Kava Social Chess Attendance Predictor

Harold Gonzalez · Distributed Systems for Data Science · NCF · Spring 2026

What it predicts

Unique-player attendance for a future Kava Social chess bracket night in Bradenton, FL. Two heads share one feature row: a **regressor** for `attendance_count` and a **classifier** for `high_turnout` (1 if attendance ≥ historical median). The model is built to plan boards, clocks, and staffing — not to forecast game outcomes.

Data source

~72 historical Kava Social bracket nights (Sep 2022 – Apr 2026), stored as game-level rows (`Date, Time, White, Black, Winner`). Each row is one chess game; one bracket night contains 20–50 such rows. I aggregate to event level — attendance for a date is the count of unique non-null players appearing in either color column. Weather is pulled from the free Open-Meteo archive + forecast APIs for Bradenton (27.50 N, –82.57 W). One-time historical load + ad-hoc refresh for new events.

Pipeline architecture

raw TSV → S3 bronze → PySpark silver/gold (S3) → scikit-learn + MLflow → Streamlit Cloud

Two distributed/cloud stages: (1) bronze ingestion to **AWS S3** (`s3://kava-chess-pipeline/bronze/`) via boto3; (2) **PySpark** transformation reading raw TSV from S3, applying name normalization + deduplication + window-function lag/rolling features, and writing **silver** (game-level Parquet) and **gold** (event-level Parquet) back to S3. A pandas mirror runs locally for fast iteration with the exact same schema. Models are trained with scikit-learn, tracked in **MLflow** (local file store), and persisted to `models/*.joblib`. The Streamlit app (hosted on **Streamlit Community Cloud**) loads the model directly — no extra serving infrastructure required.

Model approach

Random Forest Regressor (400 trees, depth 8) for attendance count and a **Random Forest Classifier** (400 trees, depth 6, class-weight balanced) for high-vs-low turnout. Features (~40 total): calendar (month, day-of-week, holiday-week, school-break flags), scheduling (`days_since_last_event`, weekly/biweekly/long-break indicators, events-this-month-so-far), prior-event lag (`previous_event_attendance`, `attendance_two_events_ago`, rolling 3/5/10 averages, trend deltas), prior-event community momentum (`previous_event_num_games`, `previous_event_new_players_count`, rolling-3 averages), and Bradenton weather (high/low/mean temperature, feels-like, precipitation, rain/thunderstorm indicators, wind). **No same-night features** are used as predictors — they would not be observable when planning the night. Evaluation is 4-fold **time-series cross-validation** so test folds are always strictly after train folds.

Reported metrics on 4-fold time-series CV over 72 events: regression **MAE = 3.67** players, **RMSE = 4.37** (attendance ranges 10–36, mean 16.3); classifier **accuracy = 64.3%**, **F1 = 0.59**. Exact values are in `models/metadata.json` and in the MLflow runs.

What I learned

Two things bit harder than expected. **First, deduplication and name normalization** — the same player appears as "Gonzalez, Harold" and "Harold" and a handful of variants, and the raw column "Winner" sometimes contains "Draw", sometimes a name, sometimes the loser by typo. I kept the merge list conservative (only confirmed mappings) so attendance counts don't get inflated by aggressive matching, and accepted that ~3 events will be slightly noisy. **Second, leakage was very easy to introduce** — every interesting same-night number (game count, draw rate, new-player count) is information you only have *after* people show up. The strict rule of "predict event E using only events strictly before E" forced me to use `pandas.shift(1)` everywhere and verify with TimeSeriesSplit instead of regular CV. If I rebuilt it I would (a) add an automated weekly retraining job (GitHub Actions + S3), (b) wire a real serving endpoint (FastAPI on Render) so the test script could call a true REST endpoint instead of the Streamlit URL, and (c) collect "did the event actually happen on Sunday or get moved?" as a feature, since holiday-week reschedules visibly distort attendance.